

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт
з лабораторної роботи №1
з дисципліни: «Поглиблене вивчення JavaScript»
з теми: «Розробка базового застосунку керування задачами (To-Do List)»

Виконав:
ст. гр. ПЗП-23-3
Білоус А. А.

Перевірив:
доц. каф. ПІ
Мар'їн С. О.

1 РОЗРОБКА БАЗОВОГО ЗАСТОСУНКУ КЕРУВАННЯ ЗАДАЧАМИ (TO-DO LIST)

1.1 Мета роботи

Ознайомлення з основними принципами роботи фреймворку Vue 3, формування практичних навичок створення односторінкових застосунків, засвоєння механізмів реактивності, директив, обробки подій та компонентного підходу.

1.2 Хід роботи

1.2.1 Структура проєкту

Проєкт побудовано з використанням Vite + Vue 3 із поділом на три компоненти:

- «App.vue» – кореневий компонент, містить стан та логіку застосунку;
- «TodoList.vue» – компонент відображення списку задач;
- «TodoItem.vue» – компонент окремої задачі.

1.2.2 Реалізація кореневого компонента App.vue

Кореневий компонент містить реактивні змінні стану: масив задач «todos», рядок нової задачі «newTask» та поточний фільтр «filter». Обчислювана властивість «filteredTodos» формує похідний список відповідно до обраного режиму відображення.

```
<script setup>
import { ref, computed } from 'vue'
import TodoList from './components/TodoList.vue'

const todos = ref([])
const newTask = ref('')
const filter = ref('all')
let nextId = 1

const filteredTodos = computed(() => {
  if (filter.value === 'active') return todos.value.filter(t => !
t.done)
  if (filter.value === 'done') return todos.value.filter(t => t.done)
  return todos.value
})

function addTask() {
```

```

const text = newTask.value.trim()
if (!text) return
todos.value.push({ id: nextId++, text, done: false })
newTask.value = ''
}

function toggleTask(id) {
  const todo = todos.value.find(t => t.id === id)
  if (todo) todo.done = !todo.done
}

function deleteTask(id) {
  todos.value = todos.value.filter(t => t.id !== id)
}
</script>

```

Шаблон кореневого компонента містить поле введення з двостороннім зв'язком даних («v-model»), кнопку додавання, кнопки фільтрації та компонент «ToDoList»:

```

<template>
  <div class="container">
    <h1>Список задач TO-DO List</h1>
    <div class="input-row">
      <input v-model="newTask" type="text"
        placeholder="Введіть задачу"
        @keyup.enter="addTask" />
      <button class="add-btn" @click="addTask">Додати</button>
    </div>
    <div class="filters">
      <button :class="{ active: filter === 'all' }"
        @click="filter = 'all'">Всі</button>
      <button :class="{ active: filter === 'active' }"
        @click="filter = 'active'">Активні</button>
      <button :class="{ active: filter === 'done' }"
        @click="filter = 'done'">Виконані</button>
    </div>
    <ToDoList :todos="filteredTodos"
      @toggle="toggleTask" @delete="deleteTask" />
  </div>
</template>

```

1.2.3 Реалізація компонента ToDoList.vue

Компонент отримує відфільтрований масив задач через props та за допомогою директиви «v-for» ітерує по колекції, відображаючи окремий компонент для кожної задачі. Атрибут «:key» забезпечує унікальність елементів.

```

<script setup>
import TodoItem from './TodoItem.vue'
defineProps({ todos: { type: Array, required: true } })

```

```

const emit = defineEmits(['toggle', 'delete'])
</script>

<template>
  <ul class="todo-list">
    <TodoItem
      v-for="todo in todos"
      :key="todo.id"
      :todo="todo"
      @toggle="emit('toggle', $event)"
      @delete="emit('delete', $event)"
    />
  </ul>
</template>

```

1.2.4 Реалізація компонента TodoItem.vue

Компонент відображає одну задачу: кнопку зміни стану, текст та кнопку видалення. Клас «done» застосовується умовно через «:class» для візуального відображення виконаних задач.

```

<script setup>
defineProps({ todo: { type: Object, required: true } })
const emit = defineEmits(['toggle', 'delete'])
</script>

<template>
  <li class="todo-item">
    <button class="toggle-btn"
      :class="{ done: todo.done }"
      @click="emit('toggle', todo.id)"></button>
    <span class="todo-text"
      :class="{ done: todo.done }">{{ todo.text }}</span>
    <button class="delete-btn"
      @click="emit('delete', todo.id)">Видалити</button>
  </li>
</template>

```

1.3 Висновки

У ході лабораторної роботи було розроблено односторінковий застосунок керування задачами (To-Do List) на основі фреймворку Vue 3 із використанням інструменту збірки Vite. Реалізовано повний базовий функціонал: додавання задач із перевіркою на порожній ввід, відображення списку через ітерацію по реактивному масиву, зміну стану задачі (виконано/активна), видалення задачі та фільтрацію за трьома режимами (всі, активні, виконані). Застосунок розподілено на три компоненти, що відповідає принципам компонентного підходу Vue.

1.4 Відповіді на контрольні запитання

- а) **Що таке SPA?** Single Page Application (односторінковий застосунок) – вебзастосунок, який завантажується одного разу і динамічно оновлює вміст сторінки без повного перезавантаження, взаємодіючи із сервером у фоновому режимі.
- б) **Основні принципи роботи Vue.js.** Vue базується на реактивній моделі даних (зміна стану автоматично оновлює DOM), компонентному підході (інтерфейс складається з ізольованих компонентів), декларативних шаблонів із директивами та однонаправленому потоці даних між компонентами через props та події.
- в) **Що таке реактивність?** Реактивність – механізм автоматичного відстеження залежностей між даними та DOM. Коли реактивна змінна (створена через «ref» або «reactive») змінюється, Vue автоматично оновлює всі шаблонні вирази, що від неї залежать, без явних викликів з боку розробника.
- г) **Яке призначення директив?** Директиви – це спеціальні атрибути шаблону з префіксом «v-», які додають реактивну поведінку до DOM-елементів. Наприклад: «v-model» – двосторонній зв'язок даних, «v-for» – ітерація по колекції, «v-if» – умовне відображення, «v-on» / «@» – обробка подій, «:class» / «:style» – динамічні атрибути.
- д) **Як реалізується обробка подій?** Обробка подій реалізується директивою «v-on» (скорочено «@»), яка прив'язує DOM-подію до методу або виразу. Наприклад, «@click="addTask"» викликає функцію «addTask» при кліку. Для дочірніх компонентів використовується система emit: дочірній компонент викликає «emit('назва-події')», батьківський прослуховує через «@назва-події».
- е) **Що таке компонент?** Компонент – це самостійна, повторно використовувана одиниця інтерфейсу, яка інкапсулює шаблон (template), логіку (script) та стилі (style). У Vue компоненти описуються у «.vue»-файлах (Single File Components). Вони спілкуються через props (дані від батька до дитини) та events (повідомлення від дитини до батька).